

SOLANA TURBIN3 Capstone Assignment 2

Capstone Idea: Solify - A Rust-based CLI Tool for Solana Developers

Name: Aditya

Turbin3 Public Key: 7qpCbG1WfLWsh4FhfoD5i5RbcVnFXuRbruG6ExhuRhe9

GitHub Profile: adisehrawat

Table of Contents

Part A: User Stories & On-Chain Requirements Document

1. Core User Personas
2. Function Maps
3. Refined User Stories
4. Potential On-Chain Requirements

Part B: Process Appendix

1. Manual User Brainstorming
2. AI-Assisted User Prioritization
3. Core Function Mapping
4. Deriving Core POC Requirements
5. Adversarial Analysis & Granularity Check
6. Final Manual Review & Refinement

Part A: User Stories & On-Chain Requirements Document

Core User Personas

1. Independent Solana Developer (Primary - Ages 18-30)

Profile: Experienced developer working on personal or freelance Solana projects who values efficiency and workflow optimization.

Pain Points:

- Constantly switching between terminal and block explorers disrupts flow
- Manual test writing is time-consuming and error-prone
- Debugging transactions requires multiple tools and context switches

Goals:

- Maintain uninterrupted development flow
 - Reduce time spent on repetitive testing tasks
 - Debug transactions quickly without leaving the terminal
-

2. Beginner Anchor Developer

Profile: New to Solana/Anchor development, learning the ecosystem and struggling with complex debugging.

Pain Points:

- Cryptic error messages are difficult to understand
- Transaction flows are opaque and hard to visualize
- Steep learning curve for on-chain program development

Goals:

- Understand what went wrong when transactions fail
 - Learn best practices through working examples
 - Build confidence in writing and testing smart contracts
-

3. Solify Power User

Profile: Developer who uses Solify regularly across multiple projects and machines.

Pain Points:

- Reconfiguring development environment on different machines
- Losing track of previous work and configurations
- No centralized history of development activity

Goals:

- Seamless environment synchronization across devices
- Persistent configuration and activity history
- Quick access to past transactions and test results

Function Maps

Independent Solana Developer Functions

Core Interactions:

- 1. Live Transaction Inspection**
 - View real-time transaction results without switching contexts
 - Monitor compute units and performance metrics
 - Inspect account state changes immediately
 - 2. Post-Execution Transaction Analysis**
 - Look up historical transactions by signature
 - Deep-dive into transaction structure and logs
 - Validate expected vs actual behavior
 - Debug failed transactions with detailed breakdowns
 - 3. Automated Test Generation**
 - Generate TypeScript test suites from Anchor IDL
 - Create both positive and negative test cases
 - Save hours on boilerplate test writing
 - Use generated tests as foundation for custom logic
-

Beginner Anchor Developer Functions

Core Interactions:

- 1. Error Understanding & Learning**
 - Receive plain-English error explanations
 - See highlighted failure points in transaction flow
 - Access remediation suggestions for common mistakes
 - Learn Anchor constraints through failed examples
- 2. Transaction Flow Visualization**
 - Understand account relationships visually
 - Follow instruction execution step-by-step
 - See how data flows between accounts
 - Learn by observing successful patterns
- 3. Guided Test Examples**
 - Study auto-generated tests as learning material
 - Understand proper account setup and initialization
 - Learn defensive programming through negative tests
 - Build confidence with working code examples

Solify Power User Functions

Core Interactions:

1. **Configuration Management**
 - Save workspace preferences on-chain
 - Store frequently used RPC endpoints
 - Maintain keypair configurations
 - Set project-specific defaults
 2. **Activity Dashboard**
 - View recent commands and transactions
 - Track test generation history
 - Monitor development patterns over time
 - Access quick links to past work
 3. **Cross-Device Synchronization**
 - Automatically load saved settings on any machine
 - Resume work context seamlessly
 - Share configurations across team members
 - Maintain consistent development environment
-

Refined User Stories

Developer Stories (Core Functionality)

Story 1: Live Transaction Execution

As a developer, I want to run a command in my program and immediately see the transaction's result in my terminal, so I can test my changes instantly without switching to a block explorer.

Story 2: Historical Transaction Lookup

As a developer, I want to look up the details of a past transaction using its signature, so I can analyze it without using a web-based explorer.

Story 3: Activity Dashboard

As a developer, I want to view a dashboard of my recent activity, like commands run and tests created, so I can easily track my work and resume where I left off.

Story 4: Simple Error Explanations

As a developer, I want to see a simple, plain-English explanation when my transaction fails, so I can quickly understand the mistake and fix it without digging through documentation.

Story 5: Forensic Transaction Analysis

As a developer, I want to view a forensically accurate breakdown of any transaction, so I can validate its precise technical details for auditing or debugging.

Story 6: Automated Test Generation

As a developer, I want to generate complete TypeScript test suites for my Anchor programs automatically, so I can save time and ensure comprehensive test coverage.

Solify Account Stories (Platform Features)

Story 7: Configuration Backup

As a Solify user, I want the tool to automatically save my settings and activity to the Solana blockchain, so my work is backed up and I never lose my configuration.

Story 8: Configuration Restoration

As a Solify user, I want the tool to automatically load my saved settings when I start it on any machine, so I can resume my work seamlessly without manual setup.

Potential On-Chain Requirements

Story 1: Live Transaction Execution

Potential On-Chain Requirements:

- Need a mechanism to build and sign transactions programmatically from the TUI
- Must securely access user's keypair for transaction signing
- Require RPC connection to submit transactions to the network
- Need to fetch and deserialize transaction confirmation data
- Must parse program logs and return data in real-time
- Require ability to decode instruction data using program IDL
- Need to display account state changes (before/after snapshots)
- Must track compute units consumed by the transaction

Story 2: Historical Transaction Lookup

Potential On-Chain Requirements:

- Need to call `getTransaction()` RPC method with transaction signature
 - Must fetch transaction in "jsonParsed" encoding format
 - Require ability to decode instruction data from raw bytes using IDL
 - Need to parse and display all account keys and their metadata (writable/signer status)
 - Must extract and format program logs for readability
 - Require retrieval of account states before and after transaction
 - Need to display transaction metadata (slot, block time, fee)
 - Must handle failed transactions and display error information
-

Story 3: Activity Dashboard

Potential On-Chain Requirements:

- Need an on-chain account to store user activity metadata
 - Must create an instruction to log transaction signatures executed by user
 - Require ability to store timestamps of user actions
 - Need to track which programs were interacted with
 - Must store references to generated test files (hash or metadata)
 - Require instruction to update activity log when new actions occur
 - Need to query and retrieve recent activity from on-chain account
 - Must implement pagination for large activity histories
-

Story 4: Simple Error Explanations

Potential On-Chain Requirements:

- Need to fetch transaction error codes from failed transactions
- Must map Anchor error codes (e.g., 0x1779) to custom program errors
- Require parsing of error enum from program IDL
- Need to extract custom error messages defined in program
- Must identify which instruction in transaction caused failure
- Require mapping of common Solana errors to user-friendly messages
- Need to provide context about which account or constraint failed
- Must display error remediation suggestions based on error type

Story 5: Forensic Transaction Analysis

Potential On-Chain Requirements:

- Need to fetch complete transaction data including all metadata
 - Must retrieve pre-transaction account states for all accounts
 - Require post-transaction account states for comparison
 - Need to decode all instruction data to raw bytes and structured format
 - Must display inner instructions (Cross-Program Invocations)
 - Require parsing of all program logs with exact ordering
 - Need to show complete account metadata (owner, lamports, data length)
 - Must display transaction signatures and signers
 - Require showing compute units consumed per instruction
 - Need to display exact error codes and error logs if transaction failed
-

Story 6: Automated Test Generation

Potential On-Chain Requirements:

- Need to read and parse Anchor IDL JSON file from target/idl/ directory
 - Must extract all instruction definitions with their parameters
 - Require parsing account constraints (mut, signer, has_one, etc.)
 - Need to identify PDAs and their derivation paths from IDL
 - Must generate code to initialize provider and program instance
 - Require creation of test wallet/keypairs for testing
 - Need to generate account setup code for each instruction
 - Must create positive test cases for each instruction
 - Require generation of negative tests for constraint violations
 - Need to generate tests for invalid signer scenarios
 - Must generate tests for missing mutable permissions
 - Require tests for incorrect account types or owners
-

Story 7: Configuration Backup

Potential On-Chain Requirements:

- Need to create a Program Derived Address (PDA) for user's configuration account
- Must implement initialize_user_profile instruction
- Require account to store user's wallet public key as identifier

- Need to store preferred RPC endpoint URL(s)
 - Must store default keypair path or configuration
 - Require storage of UI preferences (theme, layout settings)
 - Need to implement update_preferences instruction for modifications
 - Must store activity metadata (recent transaction signatures)
 - Require timestamp of last configuration update
 - Need to handle account rent exemption requirements
 - Must validate that only account owner can update configuration
-

Story 8: Configuration Restoration

Potential On-Chain Requirements:

- Need to derive user's configuration PDA from their wallet address
- Must implement fetch_user_profile query functionality
- Require ability to check if configuration account exists
- Need to deserialize configuration data from on-chain account
- Must handle case where no saved configuration exists (first-time user)
- Require validation that retrieved configuration belongs to current user
- Need to apply loaded settings to local Solify environment
- Must restore RPC endpoints, keypair paths, and UI preferences
- Require fetching of recent activity history from on-chain storage
- Need to handle version compatibility if configuration schema changes

Part B: Process Appendix

1. Manual User Brainstorming

Direct Users

- Independent Solana Developers
- Smart Contract Security auditors
- Beginners in Anchor Development

Indirect Users/Beneficiaries

- End User of Solana Dapps
- Product Owners

- Project Clients

Administrators/Moderators

- Project Moderators
- System Developer (me)

Stakeholders

- Anchor framework team
 - Solana Foundation
 - Grant Programs
-

2. AI-Assisted User Prioritization

AI Prompt

"My project's value proposition is [Solify is a Rust-based CLI tool integrated with a custom Solana smart contract that streamlines program development by combining automated testing, transaction inspection, and real-time interaction within a single terminal environment. Designed as a "Development Accelerator TUI," it eliminates the need to switch between explorers or multiple tools, allowing developers to visualize and validate transactions directly in the terminal. By leveraging solana-sdk and anchor-lang, Solify integrates seamlessly into existing workflows, supporting both online and offline testing modes for faster, more efficient iterations. Its intuitive TUI enhances the developer experience, enabling smooth navigation and insightful debugging while maintaining Rust's native performance and security. Solify's goal is to make Solana development faster, more intuitive, and less error-prone—empowering independent developers, startups, and teams in DeFi, NFT, and gaming spaces to build, test, and deploy high-quality on-chain programs with significantly reduced friction and time.].

Here is a brainstormed list of all potential user types: [Direct Users: Independent Solana Developers, Smart Contract Security auditors, Beginners in Anchor Development; Indirect Users/Beneficiaries: End User of Solana Dapps, Product Owners, Project Clients; Administrators/Moderators: Project Moderators, System Developer(me); Stakeholders: Anchor framework team, Solana Foundation, Grant Programs].

Based on the value proposition, which 2-5 of these user types are the most critical to focus on for an initial Proof-of-Concept? For each user you recommend, provide a brief rationale explaining why they are essential for proving the project's core value."

AI Output

1. Independent Solana Developers

Why They're Essential: This is your primary audience. They directly feel the pain of a fragmented workflow and are highly motivated to find tools that save time. Their validation is the most crucial test of your core value proposition—being a "Development Accelerator TUI."

How They Prove Your Value: If they adopt Solify for their projects, it proves that your tool successfully reduces the friction of switching between terminals and block explorers, leading to the "faster, more efficient iterations" you promise.

2. Beginners in Anchor Development

Why They're Essential: Your project aims to make development "more intuitive" and "less error-prone." Beginners are the ultimate litmus test for this. They struggle most with understanding transaction flows and debugging on-chain programs.

How They Prove Your Value: Success with this group validates the "enhanced developer experience" claim. If they can build, test, and debug more effectively with Solify than without it, you've proven that your tool successfully lowers the steep learning curve in the Solana ecosystem.

3. Developers in Startups & Teams

Why They're Essential: Your value proposition explicitly targets "startups, and teams in DeFi, NFT, and gaming spaces." This group works under tight deadlines and needs efficient, reliable tooling. Their needs go beyond solo development into collaborative efficiency.

How They Prove Your Value: Validating Solify with a team proves it can handle the demands of a professional, fast-paced environment. It confirms that the tool is robust enough to be integrated into a company's daily workflow, not just a hobbyist's tool.

4. Smart Contract Security Auditors

Why They're Essential: Auditors are power users who live by "transaction inspection" and "insightful debugging." While your tool isn't a dedicated auditing suite, its ability to quickly visualize and validate transactions is a core part of their initial analysis workflow.

How They Prove Your Value: Positive feedback from an auditor would be a powerful endorsement of your tool's inspection capabilities. It proves that Solify provides clear, accurate, and deep insights into on-chain activity, which is valuable for both development and security analysis.

5. Anchor Framework Team Members (as Expert Testers)

Why They're Essential: Your tool's credibility hinges on its claim to "integrate seamlessly" by leveraging anchor-lang. Who better to validate this than the experts who build and maintain Anchor itself?

How They Prove Your Value: Getting feedback from the Anchor team would provide invaluable technical validation. It confirms that your tool correctly implements Anchor's principles and genuinely enhances the workflow they've created, making Solify a true ecosystem-native tool.

Manual Action & Decision

Final Prioritized User Types:

1. Independent Solana Developers

Why They're Essential: This is my main audience mainly (18-30). They directly feel the pain of a fragmented workflow and are highly motivated to find tools that save time. Their validation is the most crucial test of your core value proposition—being a "Development Accelerator TUI."

How They Prove Your Value: Adoption of Solify proves it reduces friction between terminal and explorer use, enabling faster, more efficient development while auto-generating tests to save time during program creation.

2. Beginners in Anchor Development

Why They're Essential: Beginners are the ultimate litmus test for this. They struggle most with understanding transaction flows and debugging on-chain programs. Users can understand the flow of transactions directly in the terminal.

How They Prove Your Value: Success with this group validates the "enhanced developer experience" claim. If they can build, test, and debug more effectively with Solify than without it, you've proven that your tool successfully lowers the steep learning curve in the Solana ecosystem.

3. Anchor Framework Team Members (as Expert Testers)

Why They're Essential: My tool's credibility hinges on its claim to "integrate seamlessly" by leveraging anchor-lang. Who better to validate this than the experts who build and maintain Anchor itself?

How They Prove Your Value: Getting feedback from the Anchor team would provide invaluable technical validation. It confirms that my tool correctly implements Anchor's principles and genuinely enhances the workflow they've created, making Solify a true ecosystem-native tool.

3. Core Function Mapping

AI Prompt

"For a project with this value proposition [Solify is a Rust-based CLI tool integrated with a custom Solana smart contract that streamlines program development by combining automated testing, transaction inspection, and real-time interaction within a single terminal environment. Designed as a "Development Accelerator TUI," it eliminates the need to switch between explorers or multiple tools, allowing developers to visualize and validate transactions directly in the terminal. By leveraging solana-sdk and anchor-lang, Solify integrates seamlessly into existing workflows, supporting both

online and offline testing modes for faster, more efficient iterations. Its intuitive TUI enhances the developer experience, enabling smooth navigation and insightful debugging while maintaining Rust's native performance and security. Solify's goal is to make Solana development faster, more intuitive, and less error-prone—empowering independent developers, startups, and teams in DeFi, NFT, and gaming spaces to build, test, and deploy high-quality on-chain programs with significantly reduced friction and time.]

and focusing on these prioritized user types [1. Independent Solana Developers, 2. Beginners in Anchor Development, 3. Anchor Framework Team Members (as Expert Testers)], help map out the key functions or interactions each user would need to perform."

AI Output

Independent Solana Developers

This user wants to build faster and more reliably.

1. Terminal Transaction Inspector: They'll use this feature for rapid-fire debugging. After running a command in the Solify TUI, they'll immediately scan the transaction output to check for custom logs, confirm state changes in specific accounts, and monitor compute unit usage to optimize their program. The key value for them is eliminating the context switch to a web explorer, keeping them in a state of flow.

2. TypeScript Test Generator: This is a massive time-saver. They will run a command like `solify generate-tests` to instantly scaffold a complete test suite. They will particularly value the auto-generated negative tests (e.g., testing for incorrect signers, insufficient funds, failed constraints), as these are often tedious to write manually. They'll use the generated files as a robust foundation, adding their specific business logic on top.

Beginners in Anchor Development

This user needs a guided experience to learn and avoid common mistakes.

1. Terminal Transaction Inspector: This is their primary learning tool. When a transaction fails, the inspector won't just show an error code; it will highlight the exact step that failed and provide a simplified explanation (e.g., "Account was not initialized"). When a transaction succeeds, they will study the visual flow to understand how the accounts and program interacted.

2. TypeScript Test Generator: The generated positive tests act as a practical, working example of how to interact with their on-chain program using TypeScript. It's a real-world tutorial that's more effective than documentation. The negative tests will teach them defensive programming and the importance of Anchor's security constraints from day one.

Anchor Framework Team Members (as Expert Testers)

This user is focused on technical correctness and ecosystem best practices.

1. Terminal Transaction Inspector: They will use this for deep validation. They'll trigger complex transactions involving CPIs or intricate PDA structures and scrutinize the terminal output to ensure that every piece of data is being parsed and displayed with 100% accuracy, matching the raw output from a trusted RPC node.

2. TypeScript Test Generator: They will audit the generated code for quality and adherence to Anchor best practices. They'll check if the generated tests correctly handle complex account resolution, use the latest client libraries, and properly test all of Anchor's constraints. Their validation would confirm that Solify is a high-quality tool that promotes robust and secure development patterns.

4. Deriving Core POC Requirements

Critical User Stories for POC

1. The developer will run a command to start the Terminal Transaction Inspector.
2. The developer will run a test generation command to generate TypeScript tests for their programs.

AI Prompt

"Based on these top 1-2 critical user interactions [Developer will run command to start terminal Transaction Inspector. The developer will run a test generation command to generate typescript tests for his/her programs.], what are the key technical requirements needed to build a proof-of-concept?"

AI Output

For the Terminal Transaction Inspector

This feature is about fetching, parsing, and displaying on-chain data in a user-friendly terminal interface.

- **CLI Framework:** You'll need a Rust crate like clap to parse command-line arguments and subcommands (e.g., solify inspect <TX_SIGNATURE>).
- **TUI Library:** A library is essential for rendering the interactive terminal dashboard. ratatui is a modern and powerful choice in the Rust ecosystem for building complex terminal user interfaces.
- **Solana RPC Client:** You need the solana-rpc-client crate to connect to a Solana cluster (local or devnet) and fetch transaction details using its signature.
- **Anchor IDL Parsing:** To make the transaction data human-readable, the tool must parse the program's JSON IDL file. This allows you to decode the raw instruction data into the proper function name and arguments. The serde_json crate will be required for this.

- **Data Deserialization:** You'll need to use components from the anchor-lang crate to take the raw, base64-encoded instruction data from the transaction and deserialize it into a structured, readable format based on the IDL.

For the TypeScript Test Generator

This feature requires your tool to understand the user's program structure and generate new source code files based on it.

- **Project Discovery:** The tool needs to locate and parse the Anchor.toml file to identify the workspace programs and find the corresponding Anchor IDL JSON file in the target/idl/ directory.
 - **IDL-Driven Logic:** The core of the generator will be to programmatically read and iterate through the instructions defined in the program's IDL. The IDL provides all the necessary information: instruction names, required accounts (and their constraints like mut and signer), and arguments.
 - **Code Templating Engine:** You need a way to construct the TypeScript test files. You can use a templating engine like `tera` or even simple Rust string formatting to generate the boilerplate code for `describe` and `it` blocks, imports, and provider setup.
 - **Test Case Generation Logic:**
 - **Positive Tests:** For each instruction in the IDL, generate a test that calls it with placeholder variables for the required accounts and arguments.
 - **Negative Tests:** For each instruction, analyze the account constraints in the IDL. Generate separate tests that intentionally violate these constraints (e.g., creating a test that tries to pass a read-only account where a mutable one is required, or a non-signer where a signer is expected).
 - **File System Operations:** Your tool will need to use Rust's standard library (`std::fs`) to create a `tests` directory if one doesn't exist and write the newly generated `my-program.ts` file to disk.
-

5. Adversarial Analysis & Granularity Check

AI Prompt

"Review my core user functions/stories and requirements. Considering my project's refined value proposition, do these stories truly hit the mark? Are the requirements granular enough to map to specific technical components (e.g., database schemas, API endpoints, specific blockchain programs)? What's missing or unclear?"

AI Output

Executive Summary

Overall, your alignment is very strong. The user stories are a fantastic translation of your value proposition into human terms, and the technical requirements are a solid starting point for a Proof-of-Concept.

However, to move from a PoC plan to a true development roadmap, we need to add a layer of specificity and address a few key gaps between your high-level vision and the current plan.

Part 1: Do the stories hit the mark?

Yes, absolutely. The alignment here is nearly perfect.

- "Development Accelerator TUI" is directly reflected in the Independent Developer's story of "rapid-fire debugging" and staying in a "state of flow."
- "Intuitive TUI" and "less error-prone" is validated by the Beginner's story of using the inspector as a "learning tool" with "simplified explanations."
- "Integrates seamlessly" and "high-quality" is tested by the Anchor Team Expert's need for "100% accuracy" and adherence to "best practices."

You've successfully defined who gets value and how they perceive it. No changes are needed here.

Part 2: Are the requirements granular enough?

For a Proof-of-Concept, they are a good start. They correctly identify the major crates and components. However, they aren't yet granular enough to map to specific, buildable software components.

For the Terminal Transaction Inspector:

- **Missing Data Structures:** What does the data look like after you parse it? You should define the core Rust structs that will hold the transaction information for the TUI.
- **Specific RPC Calls:** The requirement "Solana RPC Client" can be broken down. You will specifically need to call the `getTransaction` method from the `solana-rpc-client`.

For the TypeScript Test Generator:

- **Intermediate Representation:** Before templating, your tool will create an "in-memory" representation of the tests. What does that look like? This defines the logic.
- **IDL Parsing Specifics:** You need to explicitly parse the instructions, accounts, and constraints (like `mut`, `signer`, `has_one`) within the IDL to generate the negative tests. The logic will be a series of `if/else` statements based on these constraints.

Part 3: What's missing or unclear?

1. The "Custom Solana Smart Contract" is Missing

Your value proposition mentions a "custom Solana smart contract," but it's completely absent from your user stories and technical requirements. What is its purpose? Does it store user configurations?

Does it facilitate on-chain testing? This is the single biggest ambiguity. You need a user story and a set of requirements for this core component.

2. "Real-Time Interaction" vs. Post-Mortem Inspection

Your value prop says "real-time interaction," but the user stories and requirements only describe a post-mortem inspector (solify inspect <TX_SIGNATURE>). This implies you've already run the transaction elsewhere.

Unclear: Does your TUI also need to build, deploy, and execute transactions? If so, you're missing requirements for wallet integration (handling keypairs), program building (anchor build), and transaction sending. This is a massive feature that needs to be scoped.

3. Configuration and Project State

How does Solify know which RPC endpoint to use? Which keypair to use for transactions? Which program to inspect?

Missing Requirement: You need a configuration system. This could be a Solify.toml file in the project root or a series of CLI flags (solify --keypair <PATH> --url <RPC_URL>). This is essential for usability.

4. The User-Friendly Error Mapping

The beginner's story is great: "highlight the exact step that failed and provide a simplified explanation." However, this is a non-trivial technical requirement that isn't listed.

Missing Requirement: You need a mechanism to map known Solana/Anchor program error codes (e.g., 0x1779 / ConstraintSigner) to the human-readable explanations you want to show in the TUI. This might involve creating a dedicated error-mapping module in your Rust code.

Analysis of Feedback

1. Custom Smart Contract

Issue Identified: The smart contract's purpose was missing from the original plan.

Analysis: The custom on-chain component should store user-specific Solify configurations, such as developer identity, preferred RPC nodes, and recent transaction histories. It may also facilitate lightweight "test-mode" transaction execution.

Action Taken: Added a new user story and detailed requirements for this component.

2. Configuration and Project State

Issue Identified: No defined system for configuration management.

Analysis: Usability requires persistent configuration via Solify.toml or CLI flags for project-wide consistency.

Action Taken: Added configuration management module requirements.

3. User-Friendly Error Mapping

Issue Identified: The beginner's story references simplified errors without implementation details.

Analysis: A dedicated error mapping layer should translate known Anchor/Solana error codes into human-readable messages with remediation tips.

Action Taken: Added a new module requirement under TUI logic.

4. Granularity in Technical Requirements

Issue Identified: Requirements lacked data-level detail.

Analysis: Added explicit data structure definitions, RPC call breakdowns, and IR representation for the test generator.

Refined User Stories

1. Independent Developer (Advanced User)

As an independent Solana developer, I want to inspect and debug transactions in real-time so I can identify issues without leaving my terminal and maintain my "state of flow."

- **New addition:** I can choose between "live mode" (execute & observe) and "inspect mode" (post-execution analysis).
- **New addition:** I can track my activity of generating tests and transaction history on a dashboard.

2. Beginner Developer

As a beginner developer, I want Solify to highlight the steps that failed and explain it in simple terms so I can understand common Anchor errors and learn from them.

3. Anchor Team Expert

As an experienced developer, I want 100% accurate transaction decoding and log display so I can validate instruction-level details and ensure best practices.

4. Solify User (New)

As a Solify user, I want my workspace settings and recent activity stored on-chain so I can seamlessly resume my environment across devices or sessions.

Refined Technical Requirements

A. Terminal Transaction Inspector

Specific RPC Calls:

- `getTransaction(signature, { encoding: "jsonParsed" })`
- `getConfirmedSignaturesForAddress2()`
- `getProgramAccounts()` (for live updates)

B. TypeScript Test Generator

IDL Parsing:

- Parse instructions, accounts, and constraints such as `mut`, `signer`, and `has_one`

Template Logic:

- Generate positive tests
- Generate negative tests for constraint violations

C. Custom Solana Smart Contract

Purpose: Store user profiles, preferences, and transaction history metadata.

Instructions:

- `initialize_user_profile`
 - `store_recent_activity`
 - `update_preferences`
-

6. Final Manual Review & Refinement

Refined User Stories

Developer (General User)

Story 1: As a developer, I want to run a command in my program and immediately see the transaction's result in my terminal, so I can test my changes instantly.

Story 2: As a developer, I want to look up the details of a past transaction using its signature, so I can analyze it without using a web-based explorer.

Story 3: As a developer, I want to view a dashboard of my recent activity, like commands run and tests created, so I can easily track my work.

Story 4: As a developer, I want to see a simple, plain-English explanation when my transaction fails, so I can quickly understand the mistake and fix it.

Story 5: As a developer, I want to view a forensically accurate breakdown of any transaction, so I can validate its precise technical details.

Solify Account User

Story 6: As a Solify user, I want the tool to automatically save my settings and activity to the Solana blockchain, so my work is backed up.

Story 7: As a Solify user, I want the tool to automatically load my saved settings when I start it on any machine, so I can resume my work seamlessly.

User Story Refinement Table

Before Story (Original)	After Story (Refined)	Rationale
As an independent Solana developer, I want to inspect and debug transactions in real-time so I can identify issues without leaving my terminal and maintain my "state of flow." New addition: I can choose between "live mode" (execute & observe) and "inspect mode" (post-execution analysis).	Story 1: As a developer, I want to run a command in my program and immediately see the transaction's result in my terminal, so I can test my changes instantly.	Split for Atomicity & De-Jargon: The original story combined three distinct actions. This new story focuses only on the "live mode" action. Removed jargon like "real-time" and "state of flow."
(From same story as above)	Story 2: As a developer, I want to look up the details of a past transaction using its signature, so I can analyze it without using a web-based explorer.	Split for Atomicity: This story isolates the "inspect mode" action, making it a single, concrete task for the user.
New addition: I can track my activity of generating tests and transaction history on a dashboard.	Story 3: As a developer, I want to view a dashboard of my recent activity, like commands run and tests	Clarity of Action: This story was created from a feature add-on. It's now a standalone, clear action a user can take.

created, so I can easily track my work.

As a beginner developer, I want Solify to highlight the steps that failed and explain it in simple terms so I can understand common Anchor errors and learn from them.

Story 4: As a developer, I want to see a simple, plain-English explanation when my transaction fails, so I can quickly understand the mistake and fix it.

De-Jargon & Merge Persona: Removed "Anchor errors" to be more general. The need for simple errors is universal, so the persona was broadened from "Beginner" to "Developer" to avoid redundancy.

As an experienced developer, I want 100% accurate transaction decoding and log display so I can validate instruction-level details and ensure best practices.

Story 5: As a developer, I want to view a forensically accurate breakdown of any transaction, so I can validate its precise technical details.

De-Jargon: Replaced technical terms like "transaction decoding" and "instruction-level details" with clearer, outcome-focused language ("forensically accurate breakdown").

As a Solify user, I want my workspace settings and recent activity stored on-chain so I can seamlessly resume my environment across devices or sessions.

Story 6: As a Solify user, I want the tool to automatically save my settings and activity to the Solana blockchain, so my work is backed up.

Split for Atomicity & De-Jargon: The original story combined saving and loading. This story now focuses only on the "save" action. Replaced "on-chain" with the more descriptive "on the Solana blockchain."

(From same story as above)

Story 7: As a Solify user, I want the tool to automatically load my saved settings when I start it on any machine, so I can resume my work seamlessly.

Split for Atomicity: This story focuses only on the "load" action, which is a distinct technical step from saving. This clarifies the user's experience at the start of a session.

Summary

This process appendix documents the systematic approach taken to refine Solify's user stories and technical requirements from initial brainstorming through adversarial analysis. The iterative process of AI-assisted analysis combined with manual review has resulted in clearer, more actionable user stories and granular technical requirements that directly support the project's value proposition as a Development Accelerator TUI for Solana developers.